Unified Detection of Tenant-Isolation Violations and QoS Fairness Breaches in Multi-Tenant Kubernetes

H S Jayanth (231IT024) Suvarna G (231IT077)

IT418 — Cloud Computing, course project

One pipeline for the two ways a shared cluster fails its tenants: security (isolation) and performance (noisy neighbor).

The problem: shared clusters fail tenants in two ways

Isolation violation (security)

Tenant A improperly reaches Tenant B's resources: a bad RoleBinding, a missing NetworkPolicy, a misused credential.

Fairness violation (performance)

Tenant A's workload starves Tenant B: slow responses with *no* security rule broken. The "noisy neighbor".

- A control-plane monitor sees *who asked for what*, but is blind to a CPU hog that makes no illegal call.
- An eBPF network monitor sees cross-namespace flows, but not cgroup CPU throttling.
- No tool fuses both, and none is evaluated quantitatively across both.

Base paper and the gap

Shamim, Hu, Rahman — DAST for Kubernetes (FSE '25). Dynamic testing finds thousands of CIS violations that static analysis missed.

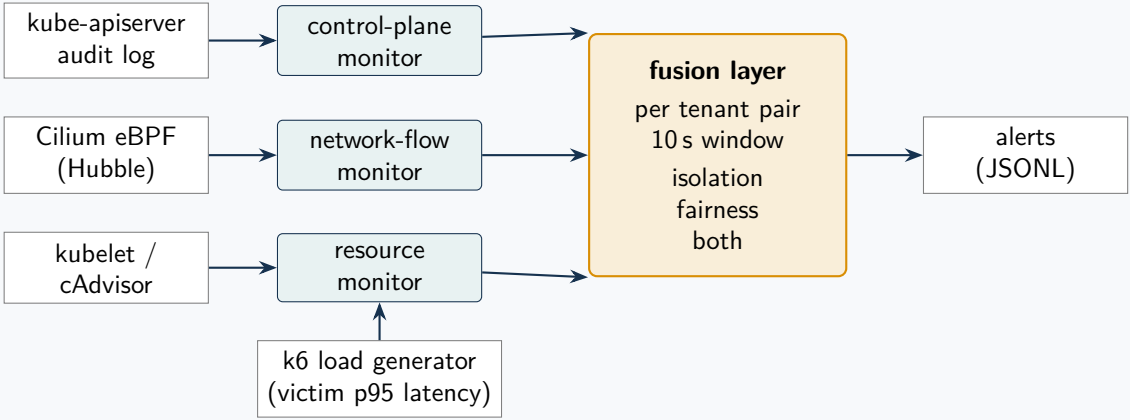
Limitations we address

- no implemented tool
- security-only; ignores fairness
- no precision / recall / latency
- control-plane signals only

Our contribution

- working multi-source pipeline
- fairness requires an independent victim-side breach
- 8 repeatable scenarios + negative controls
- honest evaluation, including what went wrong first

Architecture: three monitors, one fusion layer



Hubble's eBPF probes sit at the network layer, so resource pressure needs its own collector.

Fusion rules (deterministic, easy to validate)

1. **Isolation:** any cross-namespace control-plane or network-flow event is sufficient on its own.
2. **Fairness:** resource pressure in tenant A and a latency breach in tenant B, overlapping in time. Elevated usage alone is never enough.
3. **Both:** an isolation and a fairness cluster for the same tenant pair that overlap are merged.

Why require the victim-side signal?

A tenant legitimately using more CPU is not a violation. Without an independent measure of harm, a pressure-only rule flags every busy tenant.

Testbed and static baseline

Testbed

- kind cluster, Cilium + Hubble, API audit log
- 3 tenant namespaces, each with scoped RBAC, default-deny NetworkPolicy, ResourceQuota + LimitRange
- boundaries verified: cross-ns API call denied, flow dropped, over-quota pod rejected
- Trivy CI gate on HIGH/CRITICAL

kube-bench (CIS 1.9)

	PASS	FAIL	WARN
Master	40	7	12
Node	16	2	6
Policies	2	4	29
Total	66	13	51

It can say a control *should* be checked, not that the policy is correct and in force at runtime.

Evaluation design

8 scenarios × 5 repetitions

- *Isolation (5):* NetworkPolicy misconfig, RBAC over-permission, credential misuse, privilege escalation, data-plane-only crossing
- *Fairness (3):* CPU hog, memory pressure, quota bypass

10 negative controls

- 5 *idle* windows
- 5 *benign-busy* windows: a well-behaved tenant saturating its own small CPU limit

Idle windows can't tell naive from fused rules apart: neither fires on silence.

Detector configs: control-plane-only, network-flow-only, **fused-isolation**; pressure-only, **fused-fairness**.

Finding 1: on a big host, the noisy neighbor wasn't noisy

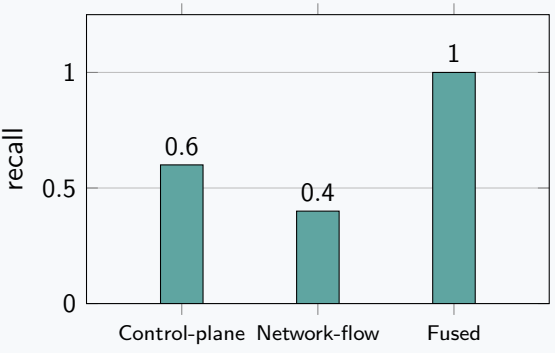
- First sweep: fused-fairness scored 0 / 15. The rule was correct; its evidence never existed.
- Replaying victim latency: p95 $\approx$ 3.6 ms *before and during* every injection. CFS-limited hogs on a 16-core host can't hurt a trivial handler.

Fix: make the contention real

Pin the worker to a 2-CPU cpuset; run hogs on the victim's node; scale them (6$\times$200m, memory-ramping pods, a multi-process quota-bypass hog).
Victim p95: ~5.7 ms $\rightarrow$ 17–95 ms.

This is engineered contention, and a result in itself: per-container CPU limits protected co-located tenants well.

Isolation: the two signals are complementary



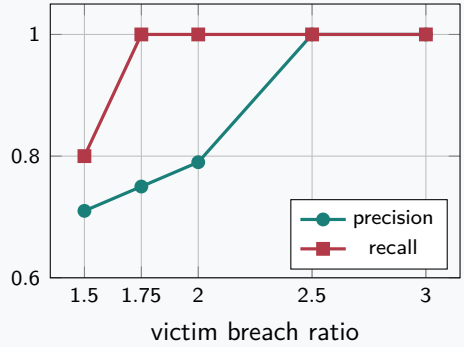
- Control-plane misses NetworkPolicy and data-plane-only runs: no API call.
- Network-flow misses RBAC, credential and privilege runs: no cross-tenant flow.
- Fused: 25/25, zero false positives.

Finding 2: idle controls hid a precision problem

Fairness config	TP/FN	FP idle/busy	Precision	Latency
pressure-only	15/0	0/3	0.83	13.5 s
fused @ 1.5×	13/2	0/5	0.72	2.5 s

- A single benign 200m job raises victim p95 to 7–9 ms vs. 5 ms idle (1.5–1.8×).
- At a 1.5× breach threshold the "independent" signal fires on busy-but-fine tenants, worse than the naive rule.
- Real noisy-neighbor runs reach 10–36 ms, so the threshold sits in the noise.

Threshold sensitivity, then validated on a fresh run



post-hoc replay of the sweep's latency data

Fresh live run at 2.5×

	pressure-only	**fused**
FP idle/busy	0/2	**0/0**
Precision	0.88	**1.00**
Recall	0.93	0.87
Latency	14.5 s	**2.5 s**

No false alarms, ∼6× faster, for a small recall cost. Threshold was chosen post hoc, so we confirmed it on independent runs.

Evaluation bugs found and fixed

- **No victim effect:** node CPU pin, co-located and scaled hogs.
- **Edge-triggered QoS events:** one event per breach onset $\Rightarrow$ re-emit while sustained; 45 s cooldown after fairness runs.
- **Merged alerts hid detections:** a "both" alert's start time came from unrelated earlier fairness evidence, scoring it a miss. Now per-track onsets: isolation 22/25 $\rightarrow$ 25/25.
- **Meaningless metric:** "correlation overhead" (about 330 s) was just time-to-scoring; replaced by measured runtime ($\approx$0.02 s).
- **Flaky tooling:** `grep -q` + `pipefail` intermittently aborted bring-up.

Limitations (stated up front)

- **2 of 15 fairness runs still missed:** resource monitor never reached its 85% threshold; or an earlier breach merged in and started before the window.
- **Post-hoc threshold**, confirmed on one fresh run; 15 runs per condition means wide intervals.
- **Engineered contention**, single host, one trivial victim workload.
- **Unaccounted alerts** aren't counted as false positives; not yet audited.
- Auto-remediation: accepted stretch goal, not implemented.

Conclusion

Takeaways

- **Isolation:** fusing control-plane and network-flow evidence detects 25/25 injections with no false positives (vs. 0.60 / 0.40 alone).
- **Fairness:** a victim-side breach requirement removes false alarms on busy tenants and detects $\sim 6\times$ sooner, at a small recall cost.
- **Honesty matters:** our first result was 0/15; the analysis that explained it is part of the contribution.

Future work: cross-validated thresholds, multi-node and realistic victim workloads, auditing unaccounted alerts, remediation.

Thank you

Questions?